Day 1 — Foundations & Linear Networks

What deep learning is, where it comes from, and the simplest network that learns

Annika Stiehl

Deep Learning • M.Sc. • SRH

Today

09:00–10:30	Welcome, getting to know each other, organisation, exam • What is deep learning? History & motivation
10:45–12:15	Perceptron & representational power • Linear regression as a neural network, gradient descent
13:00–14:30	Lab 1: Colab & PyTorch, linear regression from scratch → autograd → `nn.Module`
14:45–16:00	Lab 1 continued: first look at EEG • Project kick-off: group work & NeurIPS 2026 papers • Q&A

Welcome & Organisation

About me

- Annika Stiehl
- PhD research: deep learning for EEG seizure detection
- Interests: biomedical signal processing, robust evaluation, wearables
- *[add: affiliation, contact, office hours]*

Running theme

Many examples in this course come from my field: detecting epileptic seizures in EEG. You are free to work on images or other data in your project.

Getting to know each other

Round of introductions — about one minute each:

1. your name and your study background
2. programming: Python? NumPy? PyTorch?
3. one place where you used deep learning *this week* (knowingly or not)
4. one thing you want to take away from this course

Why this matters

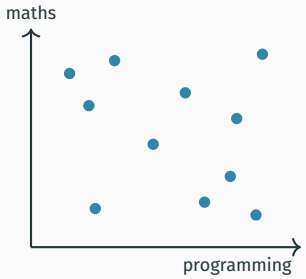
You will work in groups of 3–4 on the project for the whole module. Listen for people whose skills complement yours.

Activity: you are the training data

Stand up and place yourself in the room:

- **left** $\rightarrow$ **right**: programming experience (never $\rightarrow$ every day)
- **front** $\rightarrow$ **back**: comfort with maths (derivatives, matrices, probability)

Look around: clusters? outliers? an empty region?



Your turn

You just formed a 2D dataset. How would you choose groups of 3–4 that cover the whole space?

What you will be able to do after this course

- **Understand** how neural networks learn: loss, gradients, backpropagation, optimisation
- **Implement** MLPs, CNNs, RNNs/LSTMs, autoencoders, VAEs, GANs and GNNs in PyTorch
- **Train** them well: regularisation, normalisation, learning-rate schedules, transfer learning
- **Evaluate** them honestly: proper splits, suitable metrics, statistical tests
- **Reproduce and extend** a current research paper — and present it as paper + poster

🔑 Key takeaway

Priority in this course: understanding and coding. Every concept is derived on the slides and then implemented by you in the lab.

Course roadmap

Day	Topic	Lab	Project
1	Foundations & linear networks	PyTorch, linear regression, first EEG	kick-off
2	MLPs & backpropagation	backprop from scratch, MLP on EEG features	groups + 3 papers
3	Training deep networks & evaluation	optimisers, regularisation, leakage, statistics	proposal
4	Convolutional neural networks	CNN on CIFAR-10, 1D CNN on CHB-MIT	data pipeline
5	Transfer learning & domain shift	fine-tuning on MedMNIST	baseline check-in
6	RNN, LSTM, GRU	long memory, CNN+GRU on SeizeIT2	baseline results
7	Autoencoders & generative models	AE, VAE, EEG anomaly detection	extension, poster draft
8	Graph neural networks	GCN on Cora and EEG graphs	poster session

A typical day

- **Morning:** two lecture blocks
  intuition $\rightarrow$ math $\rightarrow$ code snippets
- **Afternoon:** lab in Google Colab
  notebook with exercises (`# TODO`)
- **From day 4:** last block partly project work,
  consultation with me

Notebooks

- `dayN_*.ipynb`: with gaps
- solutions are released after the lab
- open in Colab, select a T4 GPU

Slides & notebooks



`github.com/annikastiehl/srh-deep-learning`

Do it now (5 min)

1. scan the code, open the repository
2. click *Open in Colab* for today's notebook
3. *File* → *Save a copy in Drive*
4. *Runtime* → *Change runtime type* → *T4 GPU*
5. run the first cell — does it print `device: cuda`?

Assessment: practical work in groups (100 %)

Groups of 3–4 students:

1. pick a NeurIPS 2026 paper
2. prepare the data (CHB-MIT, SeizeIT2 or an image dataset)
3. reproduce the paper's model as **baseline**
4. develop your own **extension**
5. **evaluate** both properly (metrics + statistics)
6. upload **code** (Git repository)
7. write a **6-page short paper**
8. present a **poster** on day 8

Details

Second half of today + the project handout (`project_guidelines.pdf`).

Tools

Google Colab

- free GPU (usually an NVIDIA T4, 16 GB)
- sessions time out — save to Google Drive
- limited GPU hours per day: debug on CPU / small data first

PyTorch

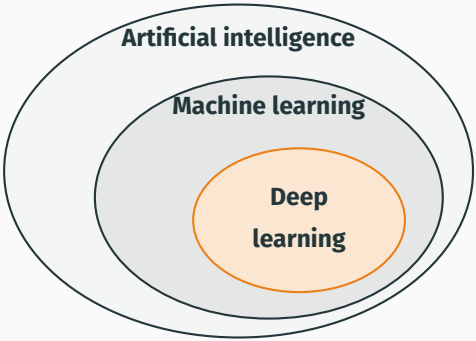
- tensors + automatic differentiation + GPU
- the de-facto standard in research code
- most NeurIPS papers release PyTorch code

Literature (all free online except Goodfellow print)

Zhang et al.: *Dive into Deep Learning* (d2l.ai) • Goodfellow, Bengio & Courville: *Deep Learning* (2016) • Nielsen: *Neural Networks and Deep Learning* • Prince: *Understanding Deep Learning* (2023) • Stanford CS231n notes

What is Deep Learning?

AI, machine learning, deep learning



- **AI**: systems that perform tasks requiring "intelligence" (also rule-based)
- **ML**: learn the mapping from *data* instead of programming it
- **DL**: ML with deep neural networks — many layers of learned, differentiable transformations

Your turn: AI, ML or DL?

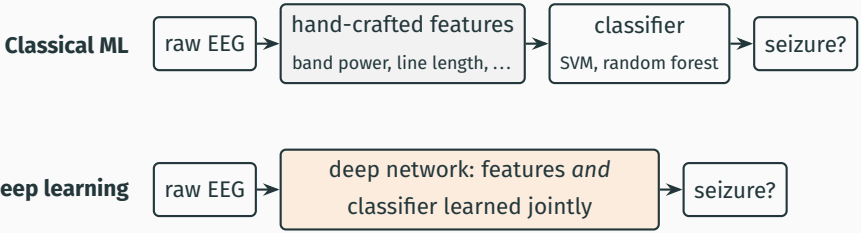
In pairs (3 min): where in the diagram does each system belong — and why?

1. Deep Blue beats Kasparov at chess (1997)
2. AlphaZero learns chess by self-play (2017)
3. route planning in a navigation app
4. seizure detection: band-power features + SVM
5. face unlock on your phone
6. a chatbot like ChatGPT or Claude

Your turn

Now place the examples from our introduction round. Which ones are you unsure about?

The key difference: learned representations



- Classical: expert knowledge goes into the *features*
- Deep learning: expert knowledge goes into the *architecture, data and loss*; features are learned
- Price: more data, more compute, less transparency

Try it: train a network in your browser

Teachable Machine —

`teachablemachine.withgoogle.com`

1. *Get started* → *Image project* → *Standard*
2. 2–3 classes, e.g. thumbs up / thumbs down / nothing; record about 50 webcam images each
3. *Train model*, then test it live
4. now break it: other background, other light, your neighbour's hand

Your turn (10 min)

Runs entirely in your browser — no GPU, no install. No webcam? Pair up, or use an *Audio project*.

Think about it

Which features did the network learn? Why does it fail when the background changes?

A short history of neural networks



- Two “AI winters” (1970s, late 1980s–90s) — the ideas are old, what changed after 2010 was data, compute and engineering
- 2024: Nobel Prize in Physics to Hopfield and Hinton for foundational work on neural networks

Why now? Three ingredients

Data

ImageNet (1.2 M labelled images), web-scale text, large clinical archives (e.g. TUH EEG corpus)

Compute

GPUs: thousands of parallel multiply-adds. Neural networks are mostly matrix products.

Algorithms & software

ReLU, dropout, BatchNorm, Adam, residual connections, autograd frameworks (PyTorch, TensorFlow, JAX)

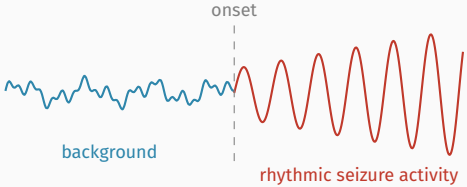
Think about it

AlexNet (2012) reduced the ImageNet top-5 error from 26.2 % to 15.3 %. Which of the three ingredients made that possible?

Krizhevsky, Sutskever & Hinton, NeurIPS 2012

Running example: epileptic seizure detection in EEG

- Epilepsy affects $\approx$ 50 million people worldwide
- Diagnosis: days of video-EEG monitoring
- Neurologists review hours of EEG by eye
- Goal: detect seizures automatically — in hospital and with wearables at home



Why this is a great deep learning problem

Long multichannel time series • extreme class imbalance (seizures are $<$ 1 % of the time) • large variability between patients • artifacts • noisy expert labels $\Rightarrow$ every topic of this course shows up.

Datasets in this course

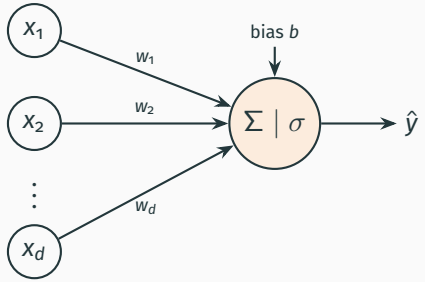
Dataset	Content	Used on
CHB-MIT	scalp EEG of 22 paediatric patients (23 cases), 23 bipolar channels, 256 Hz, 198 seizures	days 1–4, 7, 8
SeizeIT2	wearable behind-the-ear EEG (2 channels) + ECG, EMG, movement; 125 patients, 5 European centres, 886 seizures	day 6
MNIST / FashionMNIST	28×28 grey images, 10 classes	days 2, 3, 7
CIFAR-10	32×32 colour images, 10 classes	day 4
MedMNIST	small medical images (X-ray, dermatology, …)	day 5
Cora	citation graph, 2708 papers	day 8

Shoeb, PhD thesis MIT 2009 (PhysioNet) • Bhagubai et al., arXiv:2502.01224, 2025 (OpenNeuro ds005873)

The Perceptron & Representational Power

From biological to artificial neurons

- Biological neuron: dendrites collect signals, the cell “fires” if the input exceeds a threshold
- Artificial neuron: weighted sum + non-linearity
- The analogy is loose — modern networks are differentiable function approximators, not brain models



$$\hat{y} = \sigma\left(\sum_{j=1}^{d} w_j x_j + b\right) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$$

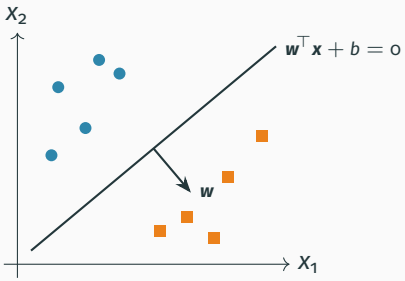
The perceptron (Rosenblatt, 1958)

Binary classifier with a step function:

$$\hat{y} = \text{sign}(\boldsymbol{w}^\top \boldsymbol{x} + b) \in \{-1, +1\}$$

Learning rule — for each misclassified sample $(y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \leq 0)$:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\, y_i \boldsymbol{x}_i, \qquad b \leftarrow b + \eta\, y_i$$



- Converges if the data are linearly separable (at most $(R/\gamma)^2$ mistakes)

Rosenblatt, Psychological Review 1958 • Novikoff 1962 (convergence proof)

Your turn: be a perceptron

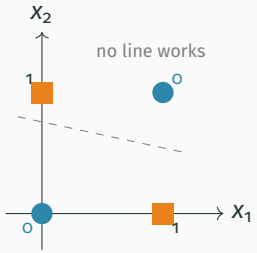
Pen and paper, in pairs (10 min). Inputs $x_1, x_2 \in \{0, 1\}$, output $\hat{y} = 1$ if $w_1 x_1 + w_2 x_2 + b > 0$, else 0.

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Your turn

1. find w_1, w_2, b for **AND**
2. find w_1, w_2, b for **OR**
3. now try **XOR**. What goes wrong?

The limits of a single neuron: XOR



- XOR is not linearly separable $\Rightarrow$ no perceptron solves it
- Minsky & Papert (1969) made this famous $\rightarrow$ interest in neural networks collapsed
- Solution: a hidden layer that transforms the input space so that the classes become separable
- But: how to train hidden layers? $\rightarrow$ backpropagation (day 2)

Think about it

Which two features $h_1(\boldsymbol{x})$, $h_2(\boldsymbol{x})$ would make XOR linearly separable?

Universal approximation theorem

Theorem (Cybenko 1989; Hornik 1991)

Let σ be a non-polynomial continuous activation (e.g. sigmoid, ReLU). For every continuous $f : K \to \mathbb{R}$ on a compact set $K \subset \mathbb{R}^d$ and every $\varepsilon > 0$ there is a network with one hidden layer

$$g(\boldsymbol{x}) = \sum_{j=1}^{N} \alpha_j \, \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j) \quad \text{with} \quad \sup_{\boldsymbol{x} \in K} |f(\boldsymbol{x}) - g(\boldsymbol{x})| < \varepsilon.$$

What it says

- Neural networks are expressive enough in principle

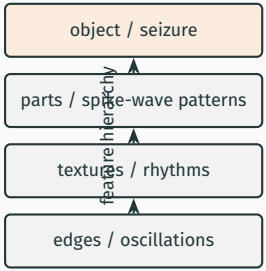
What it does *not* say

- how large N must be (can be exponential in d)
- that gradient descent finds g
- that g generalises to new data

Cybenko, Math. Control Signals Syst. 1989 • Hornik, Neural Networks 1991 • Leshno et al., Neural Networks 1993

Why deep and not just wide?

- ReLU networks are piecewise linear; the number of linear regions can grow exponentially with depth but only polynomially with width
- There are functions that a deep network represents compactly but a shallow one needs exponentially many units for
- Depth = composition: re-use of intermediate features



Montúfar et al., NeurIPS 2014 • Telgarsky, COLT 2016

Try it: XOR in TensorFlow Playground

`playground.tensorflow.org` — dataset *Exclusive or*

1. remove all hidden layers, inputs X_1, X_2 only. Press play. What happens?
2. add the input X_1X_2. Now?
3. back to X_1, X_2 only, but one hidden layer with 4 neurons. Now?
4. dataset *Spiral*: what is the smallest network that solves it?
5. set the learning rate to 3 or 10. What happens to the loss?

Your turn (15 min)

In pairs, in your browser. Note your answers; we compare afterwards.

The three big challenges (preview)

Optimisation

Deep networks are hard to train: vanishing / exploding gradients, bad initialisation, saddle points.

→ days 2, 3, 6

Overfitting

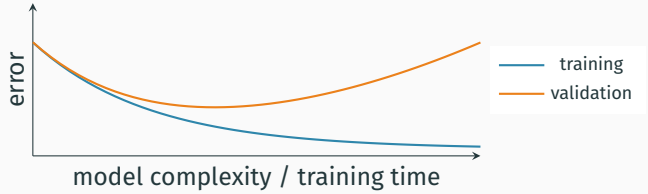
Millions of parameters can memorise the training set (even random labels!).

→ day 3: regularisation

Generalisation

Does it work on a new patient, a new hospital, a new device?

→ days 3, 5: evaluation, domain shift



Zhang et al., ICLR 2017 ("Understanding deep learning requires rethinking generalization")

Linear Regression as a Neural Network

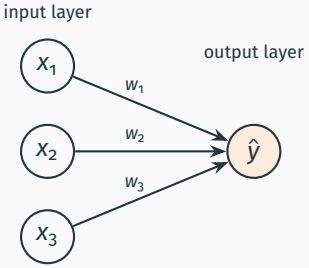
Linear regression = a single neuron without activation

Data: $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n}$, $\boldsymbol{x}_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$

Model:

$$\hat{y} = \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{j=1}^{d} w_j x_j + b$$

- Parameters $\boldsymbol{\theta} = (\boldsymbol{w}, b)$: $d + 1$ numbers
- Activation = identity
- Example: predict the age of a patient from EEG band powers



The loss: mean squared error

$$\mathcal{L}(\boldsymbol{w}, b) = \frac{1}{n}\sum_{i=1}^{n}\left(\hat{y}_i - y_i\right)^2 = \frac{1}{n}\sum_{i=1}^{n}\left(\boldsymbol{w}^\top \boldsymbol{x}_i + b - y_i\right)^2$$

Why squares? Assume Gaussian noise $y = \boldsymbol{w}^\top \boldsymbol{x} + b + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma^2)$:

$$-\log p(y \mid \boldsymbol{x}) = \frac{1}{2\sigma^2}(y - \boldsymbol{w}^\top \boldsymbol{x} - b)^2 + \text{const.}$$

$\Rightarrow$ minimising MSE = maximum likelihood

Think about it

What happens to MSE if 1% of your EEG labels contain a huge artifact value? Which loss would be more robust?

Matrix form and the closed-form solution

Stack the data: $\boldsymbol{X} \in \mathbb{R}^{n \times (d+1)}$ (with a column of ones for b), $\boldsymbol{y} \in \mathbb{R}^n$, $\boldsymbol{w} \in \mathbb{R}^{d+1}$:

$$\hat{\boldsymbol{y}} = \boldsymbol{X}\boldsymbol{w}, \qquad \mathcal{L}(\boldsymbol{w}) = \frac{1}{n}\|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|^2$$

Gradient:

$$\nabla_{\boldsymbol{w}}\mathcal{L} = \frac{2}{n}\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}) \stackrel{!}{=} \mathrm{o} \quad \Rightarrow \quad \boldsymbol{w}^* = (\boldsymbol{X}^\top\boldsymbol{X})^{-1}\boldsymbol{X}^\top\boldsymbol{y}$$

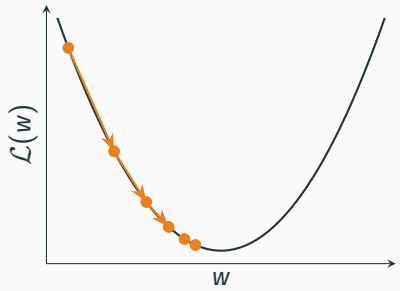
- exact, one line of code
- costs $O(nd^2 + d^3)$, all data in memory
- exists only for linear models + MSE
- deep networks: no closed form $\Rightarrow$ iterative optimisation

Gradient descent

Idea: walk downhill on the loss surface.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \, \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$$

- η: learning rate (step size)
- the gradient points to the steepest *ascent*, hence the minus sign
- stops where $\nabla \mathcal{L} = 0$ (for MSE + linear model: the global minimum, the loss is convex)



Deriving the gradient by hand

For one parameter, $\hat{y}_i = wx_i + b$, $\mathcal{L} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)^2$. Chain rule:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n}\sum_i \underbrace{2(\hat{y}_i - y_i)}_{\partial \mathcal{L}/\partial \hat{y}_i} \underbrace{x_i}_{\partial \hat{y}_i/\partial w}, \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n}\sum_i 2(\hat{y}_i - y_i)\cdot 1$$

- the error $(\hat{y}_i - y_i)$ is propagated back and multiplied with the input
- this is backpropagation in its simplest form (day 2: through many layers)

Always check with finite differences

$$\frac{\partial \mathcal{L}}{\partial w} \approx \frac{\mathcal{L}(w+\varepsilon) - \mathcal{L}(w-\varepsilon)}{2\varepsilon}$$

You will do this in the lab.

Your turn: one gradient-descent step by hand

Data ($n = 3$):

x_i	0	1	2
y_i	1	3	5

Model $\hat{y} = wx + b$, start at $w = b = 0$, learning rate $\eta = 0.1$.

Your turn (10 min, in pairs)

1. compute $\hat{y}_i$ and the loss $\mathcal{L}$
2. compute $\partial \mathcal{L}/\partial w$ and $\partial \mathcal{L}/\partial b$
3. update w and b
4. compute the new loss. Did it go down?

The learning rate



- too small: slow • too large: oscillates or diverges (loss becomes `nan`)
- for a quadratic with curvature c: converges iff $\eta < 2/c \Rightarrow$ the best η depends on the curvature

Try it: find the largest learning rate

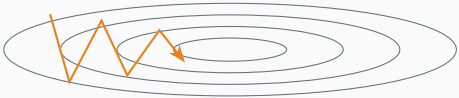
Interactive figure at the top of `distill.pub/2017/momentum` (Goh, 2017)

1. set momentum $\beta = 0$ (plain gradient descent)
2. increase the step size α until the path diverges
3. just below that value: why does the path zig-zag?

Your turn (5 min)

The loss surface is a long, narrow valley: steep in one direction, flat in the other.

Feature scaling changes the loss landscape



unscaled: zig-zag



standardised: straight to the minimum

- Features on different scales (EEG in μV vs. age in years) $\Rightarrow$ very different curvatures per direction
- One learning rate cannot fit all directions $\Rightarrow$ standardise: $x' = (x - \mu)/\sigma$
- Compute μ, σ on the training set only and apply them to validation/test data

Stochastic and mini-batch gradient descent

The full gradient needs *all* n samples: $\nabla \mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} \nabla \ell_i$ — too expensive for large n.

Mini-batch SGD: estimate it on a random batch $\mathcal{B}$ of size B:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \, \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_{\boldsymbol{\theta}} \, \ell_i(\boldsymbol{\theta})$$

- unbiased but noisy gradient estimate
- **iteration**: one update • **epoch**: one pass over the data (n/B iterations)
- noise can even help to escape bad regions

B	updates/epoch	per update
1	many	noisy, serial
32–512	medium	GPU-parallel
n	one	exact, slow

The algorithm

Mini-batch SGD

1. initialise θ
2. for each epoch:
   - 2.1 shuffle the data
   - 2.2 for each batch $\mathcal{B}$:
     - forward: $\hat{\boldsymbol{y}}_{\mathcal{B}} = f_{\theta}(\boldsymbol{X}_{\mathcal{B}})$
     - loss: $\mathcal{L}_{\mathcal{B}}$
     - backward: $\nabla_{\theta}\mathcal{L}_{\mathcal{B}}$
     - update: $\theta \leftarrow \theta - \eta\nabla_{\theta}\mathcal{L}_{\mathcal{B}}$
3. evaluate on validation data

```
model = nn.Linear(d, 1)
opt = torch.optim.SGD(model.parameters(), lr=0.01)
loader = DataLoader(train_ds, batch_size=64,
                    shuffle=True)
for epoch in range(epochs):
    for xb, yb in loader:
        loss = F.mse_loss(model(xb), yb)  # forward
        opt.zero_grad()                   # reset
        loss.backward()                   # backward
        opt.step()                        # update
```

This loop is the same for every model in this course.

Multiple outputs: a linear layer

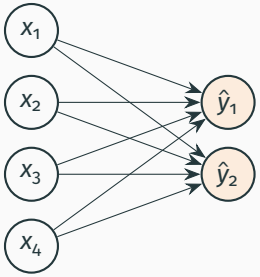
Predict k targets at once:

$$\hat{\boldsymbol{y}} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \qquad \boldsymbol{W} \in \mathbb{R}^{k \times d},\ \boldsymbol{b} \in \mathbb{R}^{k}$$

For a mini-batch $\boldsymbol{X} \in \mathbb{R}^{B \times d}$ (one sample per row):

$$\hat{\boldsymbol{Y}} = \boldsymbol{X}\boldsymbol{W}^{\top} + \boldsymbol{b} \ \in \mathbb{R}^{B \times k}$$

- exactly `nn.Linear(d, k)`: `weight` has shape (k, d)
- one matrix product per batch $\Rightarrow$ fast on GPUs



fully connected: $k \cdot d$ weights

Weight initialisation

- Linear regression with MSE is convex: any initialisation (even zeros) reaches the same minimum
- With hidden layers (day 2) this breaks down:
  - all weights equal $\Rightarrow$ all hidden units compute the same thing and receive the same gradient $\Rightarrow$ they stay identical forever (symmetry)
  - $\Rightarrow$ initialise *randomly* to break the symmetry
- The scale matters: too large $\rightarrow$ activations explode, too small $\rightarrow$ signals vanish
- PyTorch already uses a sensible default (Kaiming-uniform for `nn.Linear`); day 3: Xavier & He initialisation

Think about it

You initialise every weight of a 2-layer network with 0.5. After 1000 training steps, how many *different* hidden units does your network have?

Summary of the morning

Key takeaway

- Deep learning = learning representations with many differentiable layers
- One neuron = weighted sum + non-linearity; one neuron alone cannot solve XOR
- Linear regression is the simplest neural network: `nn.Linear` + MSE
- Gradient descent: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla \mathcal{L}$; mini-batches make it scale
- Learning rate, feature scaling and initialisation decide whether training works

Lab 1: PyTorch & Linear Networks

Lab 1

`day1_linear_networks.ipynb`

Tasks

1. tensors, broadcasting, GPU
2. linear regression in NumPy, gradient check
3. gradient descent, learning-rate experiment
4. the same with autograd
5. `nn.Linear` with 2 outputs, batch sizes
6. first look at a seizure in CHB-MIT

Getting started

1. open the notebook in Colab
2. *Runtime* → *Change runtime type* → *T4 GPU*
3. work in pairs, run every cell
4. stuck > 10 min? Ask!

Autograd in five lines

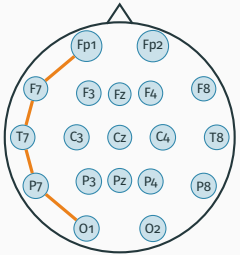
```
w = torch.tensor(0.5, requires_grad=True)   # track operations on w
b = torch.tensor(-1.0, requires_grad=True)
loss = ((w * x + b - y) ** 2).mean()        # builds a computation graph
loss.backward()                             # dL/dw, dL/db via the chain rule
print(w.grad, b.grad)                       # compare with your hand-derived gradient!
```

Two classic bugs

- forgetting `opt.zero_grad()` → gradients accumulate over steps
- updating parameters outside `torch.no_grad()` → the update becomes part of the graph

EEG in one slide

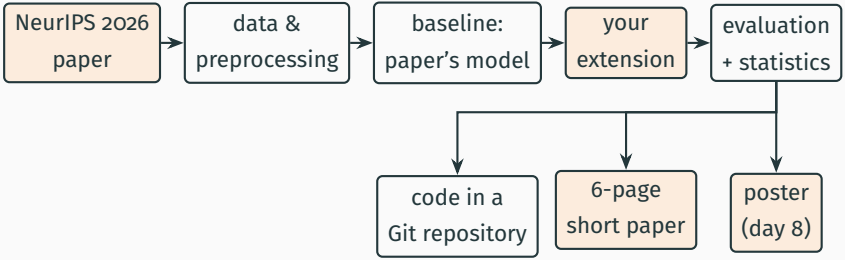
- electrodes on the scalp (10–20 system) measure voltage fluctuations ($\sim$10–100 μV)
- **bipolar montage**: differences of neighbouring electrodes, e.g. FP1 - F7
- typical rhythms: delta ($<$4 Hz), theta (4–8), alpha (8–13), beta (13–30 Hz)
- seizure: evolving rhythmic activity, often higher amplitude, spreading over channels



left temporal chain of the “double banana”

The Group Project

The project in one picture



- groups of **3–4 students**, every member contributes to code *and* paper (contribution statement)
- this mirrors real research: reproduce first, then improve, and report honestly

Step 1: choose a NeurIPS 2026 paper

Where to look

- official list of accepted papers (JSON, link in the handout): main track + *Evaluations & Datasets* track
- OpenReview forum pages (public around camera-ready)
- arXiv / GitHub: search “NeurIPS 2026” + keyword
- keywords: EEG, seizure, biosignal, time series, medical imaging, small data, robustness, …

Selection criteria

- uses — or can be applied to — CHB-MIT, SeizeIT2 or an image dataset
- code available (strongly recommended)
- one training run $\lesssim$ 1 h on a Colab T4
- a clear task, metric and baseline
- you understand the method after one careful reading

Think about it

A paper reports results of a 1B-parameter model trained on 64 GPUs. Can you still use it? (Hint: fine-tune a released checkpoint, or apply the *idea* to a smaller model.)

Datasets for the project

	What	Why interesting
CHB-MIT	22 paediatric patients, 18+ bipolar scalp channels, PhysioNet (open)	the classic benchmark; many papers to compare with; patient-specific vs. cross-patient
SeizeIT2	125 patients, wearable 2-channel behind-the-ear EEG + ECG, EMG, movement; 5 centres; OpenNeuro ds005873 (CC0)	realistic wearable setting, multimodal, cross-centre shift, seizure types, sleep/wake labels
Images	e.g. MedMNIST, CIFAR-10/100, STL-10, Oxford-IIIT Pets, …	fast iteration, many NeurIPS methods start here

Compute budget

Use a subset (e.g. a few patients) if needed — but say so in the paper and split by patient/record.

Ideas around seizure detection (for your extension)

- **Wearables**: how much do we lose with 2 instead of 18 channels?
- **Multimodal**: EEG + ECG + movement (SeizeIT2)
- **Generalisation**: train in one centre, test in another
- **Personalisation**: fine-tune on a few minutes of a new patient
- **Imbalance**: losses, sampling, event-based evaluation
- **Robustness to artifacts**: movement, chewing, electrode pops
- **Efficiency**: small models that run on a device
- **Self-supervised pretraining** on unlabelled EEG
- **Seizure types** or **sleep vs. wake** (SeizeIT2 annotations)
- **Explainability**: which channel/frequency drives the decision?

Image projects: augmentation, transfer learning, robustness to corruptions, calibration, small-data learning, …

What counts as an extension?

Good extensions

- a motivated change: “the paper ignores X, so we add Y”
- new dataset / setting for the method (e.g. scalp $\rightarrow$ wearable)
- architecture, loss, augmentation, pretraining, post-processing
- an ablation that shows which part helps

Not enough

- “we trained longer”
- random hyper-parameter changes without a hypothesis
- one lucky seed

Negative results are fine

An honest, well-evaluated “it did not help, and here is why” is a good project.

Evaluation and deliverables

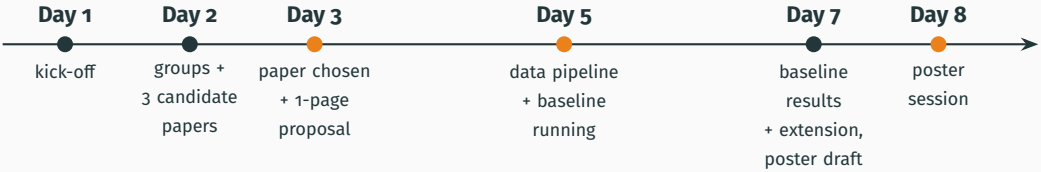
Evaluation (details: day 3)

- held-out test set, split by patient/record
- metrics that fit the task (seizures: sensitivity, false alarms/h, AUPRC, …)
- several seeds / folds, statistical test baseline vs. extension

Deliverables

- Git repository: README, environment, scripts to reproduce every number
- short paper: 6 pages in the NeurIPS template (+ references)
- poster presented on day 8
- contribution statement per member

Milestones



- after day 8: submission of short paper + code (deadline announced by the lecturer)
- proposal (day 3): paper, dataset, preprocessing, split, metrics, statistical test, compute estimate, extension idea

Common pitfalls (learn from others' mistakes)

- data leakage: windows of the same patient in train and test
- tuning on the test set
- accuracy on imbalanced data
- comparing to numbers from a different split
- starting to write the paper in the last week
- underestimating data preprocessing (it takes the longest!)
- no fixed seeds / versions $\Rightarrow$ results cannot be reproduced
- one person does all the code

EEG corner

Random window splits on CHB-MIT can give near-perfect scores that collapse on a new patient. We will measure this effect ourselves on day 3.

Wrap-up

Today

Key takeaway

- DL learns representations; the ideas are old, data + GPUs + software made them work
- Perceptron $\rightarrow$ XOR problem $\rightarrow$ hidden layers; UAT: expressive, but no free lunch
- Linear regression = one linear layer + MSE, trained by (mini-batch) gradient descent
- The PyTorch training loop: forward $\rightarrow$ loss $\rightarrow$ `zero_grad` $\rightarrow$ backward $\rightarrow$ step

Self-study & project milestone

Read / watch

- D2L, chapters 2 (preliminaries) and 3 (linear neural networks for regression)
- Nielsen, chapter 1
- 3Blue1Brown, *Neural networks*, videos 1–2 (3blue1brown.com)
- Seeing Theory, chapter *Regression Analysis* (interactive, seeing-theory.brown.edu)
- finish the lab notebook
- all links: course repository (QR code)

Project milestone for day 2

- form a group of 3–4
- browse NeurIPS 2026 papers
- bring 3 candidate papers (title, dataset, code link)

Questions?